

# ARENA 3.0 Extended: Review Packet

Local verification packet for the ARENA Frontier Lab extension

2026-07-02

## Verdict

This branch is now an append-only ARENA 3.0 extension with 41 sections marked **course\_ready**. The original ARENA material is preserved as the canonical base and is checked by an original-preservation audit. The extension adds modern architecture, sparse-feature, activation-to-language, automated circuit, alignment, representation-geometry, vision-language, image-generation, world-model, adapter, Shapley, and training-dynamics notebooks. The local release gate regenerated all 41 extension verification reports on the RTX 5090 Laptop GPU, accepted all 41, and finished with **STRICT\_COMPLETION=PASS**.

The strongest honest statement is: the extension now follows the ARENA-style course contract defined for this fork, with real CUDA-backed evidence and no mock, fake, placeholder, or fallback-only claims counted as course-ready. It is still appropriate for a human reviewer to judge prose polish and pedagogical taste against original ARENA, because an audit can enforce structure and evidence but cannot fully replace editorial judgment.

## 1 Motivation And Research Question

Can this fork extend ARENA 3.0 into a local-first frontier interpretability course without damaging the original course, diluting the exercise style into survey prose, or replacing real results with smoke tests and placeholder plots?

The answer in this packet is scoped:

- Original ARENA is preserved and remains first in the merged course configuration.
- New sections must start with toy or generated ground truth before escalating to pretrained model claims.
- Each course-ready section has one core question, one signature result, expected outputs, solution/help/interpretation dropdowns, limitations, and anomaly-hunting follow-ups.
- Hard exercises must expose sub-function tests, toy or reference oracles, controls, and a committed verification report.
- Reports are evidence, not the course itself; notebooks and pages still have to read as learner-facing ARENA material.

## 2 Repository And Review Bundle

The core review artifacts are:

- `FOR_REVIEW/FOR_REVIEW.pdf`: this human review packet.
- `FOR_REVIEW/FOR_REVIEW.tex`: LaTeX source for the packet.

- `FOR_REVIEW/ARENA_core_chapters_exercises_solutions.tar.gz`: a narrow course archive containing only chapter instruction pages and paired exercise/solution notebooks.

The archive intentionally excludes verification reports, tests, Python support files, caches, external model artifacts, Git metadata, environment files, and generated expected-output assets. Its purpose is quick pedagogical review of the core chapter and notebook surface, not reproduction of the whole repo.

### 3 Dataset And Course Structure

This packet does not review a single tabular dataset. The review object is the course corpus itself: chapter instruction pages, paired exercise notebooks, paired solution notebooks, and their verification artifacts. The companion archive contains the human-facing subset of that corpus: 78 instruction pages, 77 exercise notebooks, and 77 solution notebooks. The full repository remains the reproducibility source for tests, helper modules, expected outputs, and verification reports.

### 4 What Was Added

| Track                         | Sections | Signature learning progression                                                                                                                                                                                             |
|-------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Skepticism and infrastructure | 0.6, 1.6 | Reject fake interpretability results with leakage, cherry-pick, probe-overfit, and random-direction controls; then prove the local uv, CUDA, VRAM, parity, and activation-store contracts before trusting later notebooks. |
| Modern architectures          | 5.1–5.6  | Implement Gemma-style blocks, feature steering, Mamba scans and state tracking, toy diffusion language models, DiffusionGemma local-proof boundaries, embedding retrieval, and function-calling controls.                  |
| Sparse feature methods        | 6.1–6.4  | Compare SAE variants, inspect released Gemma Scope artifacts, build transcoders and attribution graphs, and use crosscoders for paired-model diffing.                                                                      |
| Activation to language        | 7.1–7.5  | Move from logit/tuned/attention lenses to feature verbalizers, activation oracles, natural-language autoencoders, and predictive concept decoders with held-out and random-control checks.                                 |
| Automated circuits            | 8.1–8.5  | Rebuild activation patching, attribution patching, EAP, ACDC metrics, circuit tracing, and sparse feature circuits with completeness, faithfulness, minimality, and same-size random controls.                             |
| Alignment interpretability    | 9.1–9.4  | Study refusal directions, chain-of-thought faithfulness, benign proxy drift, and white-box monitors using no-generation or aggregate-safe paths where appropriate.                                                         |

| Track                              | Sections        | Signature learning progression                                                                                                                                                                                        |
|------------------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Capstone and geometry              | 10.1, 11.1      | Run a mini research sprint and geometry-control lab with held-out templates, ablation, counterfactual patching, PCA/SVD, UMAP, white-noise, and seed stability controls.                                              |
| Vision, images, and world models   | 12.1–14.1       | Teach CLIP/SigLIP/VLM controls, diffusion/image-generation attribution, and JEPA/world-model latent prediction using controlled scenes before real-model claims.                                                      |
| Adapters and attribution baselines | 15.1, 16.1–16.8 | Compare LoRA/DoRA/full finetuning on generated proxy tasks; build exact Shapley, KernelSHAP, PartitionSHAP, interaction, TokenSHAP, VLM-region SHAP, SHAP-vs-patching, Data Shapley, and agreement/disagreement labs. |
| Training dynamics                  | 17.1            | Save and reload checkpoints, detect stable mechanism emergence, compare phase-transition timing, and reject random-label checkpoint controls.                                                                         |

## 5 Methodology: How Sections Are Graded And Tested

Course-ready status is not assigned from a passing notebook cell alone. The acceptance ladder is:

1. **Student surface.** The instruction page and notebooks must state one core question, give expected outputs, include help and interpretation dropdowns, expose solution dropdowns, and end with limitations plus anomaly hunting.
2. **Sub-function tests.** Hard implementation steps have tests for individual functions before the final result is trusted.
3. **Toy or reference oracle.** New topics start from ground-truth games, generated tasks, exact coalition tables, analytic rules, or reference model parity before real-model claims.
4. **Controls.** Every plot or headline metric must include a control: shuffled labels, random directions, random activations, text-only baselines, same-size random patches, copy baselines, mismatched partitions, or equivalent negative controls.
5. **Verification report.** Each section has a committed `verification_report.json` recording accepted metrics, claim scope, GPU evidence, known failures, and paths.
6. **Global audits.** The repo-level gates check course surface, original ARENA preservation, style depth, hard-exercise ladders, report-evidence contracts, artifact hygiene, no minified files, roadmap completion, and strict completion.

### Pseudocode

```
for section in extension_sections:
    require original_arena_preserved()
    require one_core_question(section)
    require one_signature_result(section)
    require expected_outputs_and_dropdowns(section.notebooks)
```

```

require hard_exercise_subfunction_tests(section)
require toy_or_reference_oracle_before_real_model_claim(section)
require negative_controls_for_plots_and_metrics(section)
require accepted_verification_report(section)

run hosted_cpu_gate()
run full_cuda_release_gate(max_vram_gb=24)
accept only if strict_completion_passes()

```

## 6 Results: Representative Signature Results

| Section   | Signature result                                                                                                                                       | Why it is meaningful                                                                                                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 0.6       | All four bogus interpretability claims are rejected: label leakage, cherry-picking, probe overfit, and weak random-direction claims.                   | The detectors are input-driven and include positive controls, so the result is not a hard-coded default-fixture pass. |
| 1.6       | Local readiness checks pass: uv, CUDA, BF16, VRAM, parity, and activation-store metadata.                                                              | Later model claims are blocked unless the intended runtime and evidence path are actually present.                    |
| 5.1–5.4   | Gemma and Mamba implementations match reference/parity targets and expose cache, scan, state, and intervention controls.                               | Students implement the mechanism first, then verify against known behavior.                                           |
| 8.1–8.5   | Patch, attribution, ACDC, graph, and sparse-feature circuit methods recover target positions or features while random/same-size controls fail.         | Circuit claims are tied to causal or attribution evidence, not visually pleasing graphs alone.                        |
| 12.1–14.1 | Controlled images/videos produce localized retrieval, patching, diffusion, and world-model results before broader real-model claims.                   | The visual sections reject white-noise or generic pretty outputs by requiring localized metrics and controls.         |
| 16.1–16.8 | Exact ground-truth Shapley and interaction games are recovered before sampling, VLM, activation-patching, or data-attribution comparisons are trusted. | Attribution methods are taught against mathematical objects where correctness is auditable.                           |

## 7 Qualitative Examples

The qualitative review target is not a single generated answer. It is whether the learner experience repeatedly follows the ARENA pattern: a concrete question, a small implementation, an immediate test, an expected output, a control, and an interpretation. Representative examples include:

- Section 0.6 asks students to reject plausible-looking interpretability claims, then shows why each claim fails under a targeted control.

- Section 8.5 asks students to build sparse feature circuits only after exact patching and same-size random graph controls are in place.
- Section 16.1 asks students to compute exact Shapley values on a finite game before trusting any sampled or model-based attribution shortcut.

## 8 Metrics Primer

The primary metrics are course-readiness and evidence metrics, not a single classifier score. Accuracy, reconstruction error, logit-difference recovery, Shapley error, held-out probe performance, random-control margins, and peak VRAM appear in the section reports where appropriate. MCC is not a primary metric for this packet because the packet is not evaluating one binary classification dataset; individual sections use task-appropriate metrics and controls instead.

## 9 CUDA And Environment Evidence

The final local release run was:

```
BNB_CUDA_VERSION=130 uv run python scripts/ci_quality_gate.py --mode release
```

It performed a real CUDA preflight with `nvidia-smi`, confirmed `torch.cuda.is_available()`, re-generated all 41 extension reports with a 24 GiB VRAM limit, accepted all 41, then ran the GPU-report, report-evidence, roadmap-final-completeness, and strict-completion audits. The final line was **STRICT\_COMPLETION=PASS**.

| Item                                                | Observed value                     |
|-----------------------------------------------------|------------------------------------|
| Python                                              | 3.14.6                             |
| PyTorch                                             | 2.12.1+cu132                       |
| torchvision                                         | 0.27.1+cu132                       |
| PyTorch CUDA runtime                                | 13.2                               |
| Driver CUDA UMD reported by <code>nvidia-smi</code> | 13.3                               |
| GPU                                                 | NVIDIA GeForce RTX 5090 Laptop GPU |
| Visible VRAM                                        | 23.4597 GiB                        |
| Extension reports regenerated                       | 41                                 |
| Extension reports accepted                          | 41                                 |
| CUDA-backed reports audited                         | 43                                 |

One package caveat remains: `bitsandbytes` 0.49.2 does not ship `libbitsandbytes_cuda132.so`. Local `bitsandbytes` CUDA paths therefore use `BNB_CUDA_VERSION=130` with the package's CUDA 13.0 binary. This is not represented as native CUDA 13.2 support for `bitsandbytes`.

## 10 Archive Contents

The companion tarball is deliberately small in scope. It contains:

- 78 chapter instruction-page Markdown files from `chapter*/instructions/pages/*.md`.
- 77 exercise notebooks matching `chapter*/exercises/**/*_exercises.ipynb`.
- 77 solution notebooks matching `chapter*/exercises/**/*_solutions.ipynb`.

It contains no verification reports, expected-output images, tests, Python support modules, hidden caches, downloaded models, LaTeX files, or Git metadata. This makes the archive suitable for quick review of the learner-facing course surface while keeping the full repo as the source of reproducibility evidence.

## 11 Limitations, Blockers, And Deferrals

- There are no remaining non-`course_ready` extension pages tracked in `docs/arena_style_rewrite_status.yml`.
- Huge-model routes that exceed the 24 GiB local GPU remain deferred until they can be validated honestly. They are not replaced with mock results.
- GitHub hosted CI passes CPU audits, but the self-hosted CUDA job is skipped unless a CUDA runner is attached. The CUDA evidence in this packet was run locally.
- Some sections intentionally make narrow claims, for example a one-prompt local-generation proof or a controlled generated-scene result. The claim boundary is part of the course material and should not be inflated into broad model-quality or mechanism-generalization claims.
- Automated checks can enforce structure, evidence, and known failure modes. They cannot fully replace a human review of whether the writing has the elegance, pacing, and taste of original ARENA.

## 12 Reviewer Questions

1. Does the extension feel like original ARENA in progression: implement, test, interpret, control, then escalate?
2. Are any sections still too survey-like despite having a core question and signature result?
3. Are the toy ground-truth tasks strong enough bridges to the real-model claims?
4. Which signature results are visually or pedagogically weak even if the verification report passes?
5. Are any deferred huge-model paths too important to leave as optional follow-up work for a 24 GiB local course?

## A Artifact Index

- `Extension-Roadmap.md`: final roadmap and scope.
- `README.md`: course overview and section list.
- `docs/arena_style_rewrite_status.yml`: course-ready status ledger.
- `scripts/ci_quality_gate.py`: hosted, GPU, and release gate wrapper.
- `scripts/audit_extension_completion_strict.py`: strict completion audit.
- `scripts/audit_original_arena_preservation.py`: original ARENA preservation audit.
- `chapter*/exercises/*/verification_report.json`: per-section evidence reports.
- `FOR_REVIEW/ARENA_core_chapters_exercises_solutions.tar.gz`: learner-facing chapter and notebook archive.

## B Rebuild Commands

```
tectonic FOR_REVIEW/FOR_REVIEW.tex --outdir FOR_REVIEW
pdftotext FOR_REVIEW/FOR_REVIEW.pdf -
```

```
find chapter* -path '*/instructions/pages/*.md' -type f -print
find chapter* -path '*/exercises/*' -type f \
  \( -name '*_exercises.ipynb' -o -name '*_solutions.ipynb' \) -print
```

```
BNB_CUDA_VERSION=130 uv run python scripts/ci_quality_gate.py --mode hosted-cpu
BNB_CUDA_VERSION=130 uv run python scripts/ci_quality_gate.py --mode release
```